DAY 5 – LAB 5.1

ASP.NET Core Order Application on Azure App Service

1. Business Scenario

Your organization wants to launch a web application where employees can view and submit customer orders.

The development team has developed an ASP.NET Core application using C# and .NET 10. The cloud operations team is responsible for providing the hosting environment, deploying the application and ensuring that it remains accessible.

Business objective: Deploy a customer-facing web application using managed Azure hosting without maintaining the underlying virtual machines and web server infrastructure.

What we will accomplish:

- Understand managed web application hosting.
- Build and validate an ASP.NET Core application.
- Provision Azure App Service infrastructure.
- Deploy the application to Azure.
- Configure application settings.
- Monitor and troubleshoot application availability.

2. Architecture & Technology Overview

Application architecture

User / Browser → Azure App Service → ASP.NET Core Application → HTTP Response

Technologies used

Technology	Role in our application
C#	Programming language used to implement application logic.
.NET 10	Development platform and runtime.
ASP.NET Core	Framework used to build the web application.
Razor Pages	Page-based approach for building the application interface.
Azure App Service	Managed cloud hosting platform.
App Service Plan	Provides hosting capacity and pricing tier.
VS Code	Development and deployment interface.

Understanding Azure App Service

Azure App Service is a Platform as a Service (PaaS) offering used to host web applications and APIs.

In a traditional deployment, an organization might provision a virtual machine, install an operating system, configure a web server, install the application runtime and maintain the infrastructure.

With App Service, Azure manages the underlying hosting infrastructure, allowing the development and operations teams to focus on application deployment, configuration and availability.

Common enterprise use cases:

- Customer-facing web portals.
- Internal employee applications.
- Business applications and dashboards.
- REST APIs consumed by mobile and frontend applications.
- Enterprise application backends.

When would we choose App Service?

When we need managed hosting for a conventional web application or API without taking responsibility for managing the underlying server infrastructure.

Understanding the App Service Plan

An App Service Plan defines the hosting resources, pricing tier and scaling capabilities available to the Web Apps associated with it.

Think of it this way:

- App Service Plan = Hosting capacity.
- Web App = Application hosted on that capacity.

Multiple Web Apps can share the same App Service Plan and its resources.

Why Standard S1?

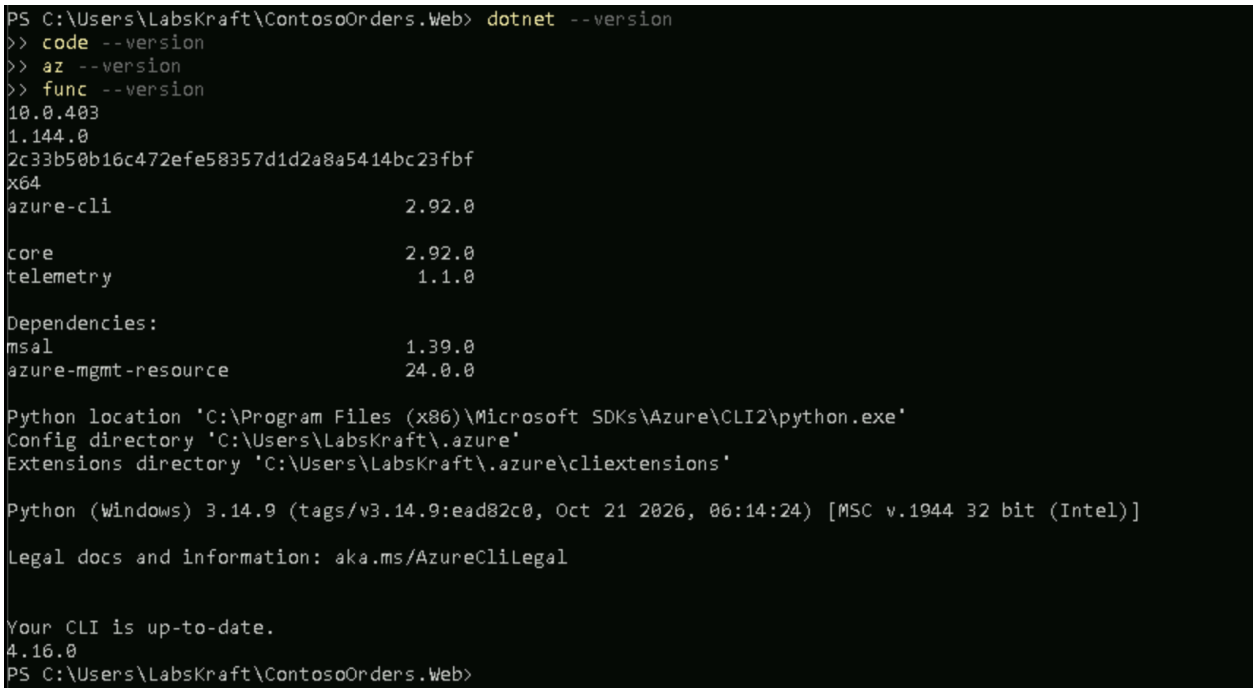
Our subsequent Lab 5.2 requires deployment slots. Standard S1 provides the required deployment-slot capability, allowing us to reuse the same hosting plan.

Free, Shared and Basic tiers do not support deployment slots.

This is a deliberate architectural and cost decision, not just a portal configuration choice.

3. Prerequisites

- Azure subscription access and resource creation permissions.
- .NET 10 SDK.
- VS Code.
- Azure App Service extension for VS Code.
- Azure account signed in through the VS Code extension.



4. Lab Execution

Phase 1 – Build the ASP.NET Core Application

Concept – What are we building?

ASP.NET Core is a web application framework that allows developers to build web applications using C#.

We are using Razor Pages, which provides a page-oriented application structure. Each page can contain its presentation markup and associated server-side logic.

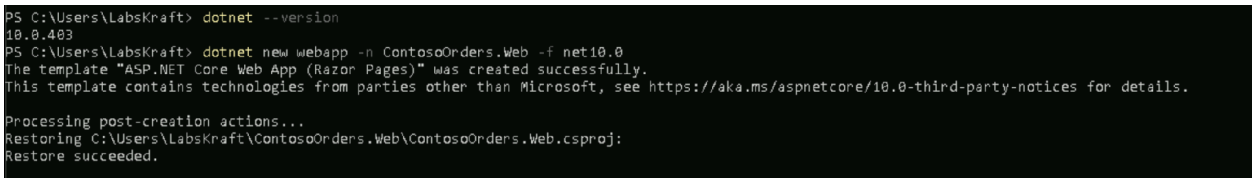
Real-world relevance: This approach is commonly used for business portals, internal applications and form-based web applications.

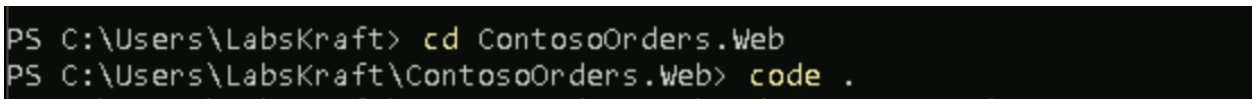
Execution

Open VS Code → Terminal → New Terminal.

Execute:

```
None
dotnet new webapp -n ContosoOrders.Web -f net10.0
cd ContosoOrders.Web
code .
```

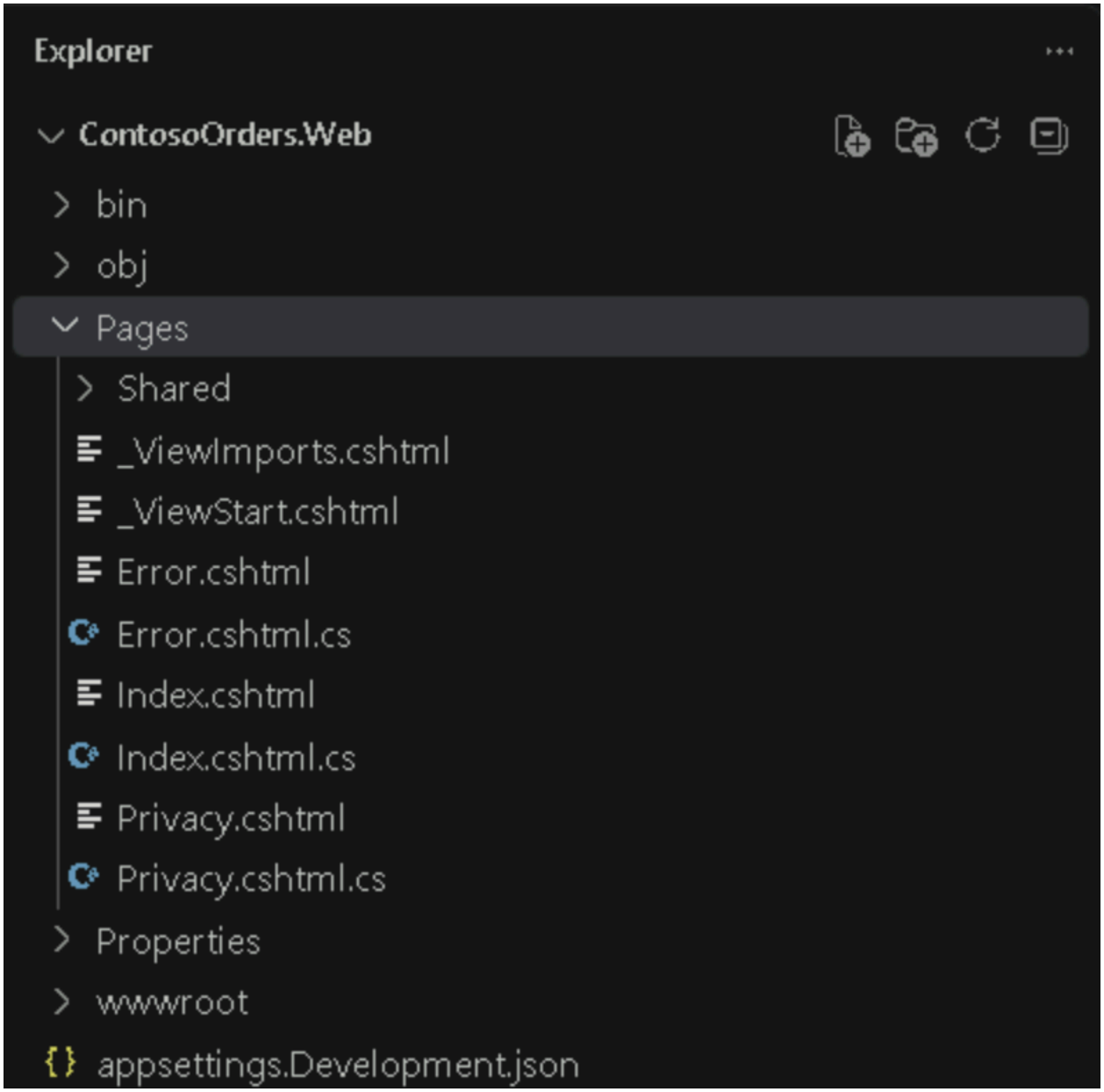




Understand the project structure

None

```
ContosoOrders.Web/
├── Pages/
│   ├── Index.cshtml
│   ├── Index.cshtml.cs
│   ├── Privacy.cshtml
│   └── Shared/
├── wwwroot/
├── Program.cs
├── appsettings.json
├── appsettings.Development.json
└── ContosoOrders.Web.csproj
```



File / Folder	Responsibility
Pages	Application pages and server-side logic.
wwwroot	Static files such as CSS, JavaScript and images.

Program.cs	Application entry point and service configuration.
appsettings.json	Application configuration defaults.
appsettings.Development.json	Development-specific configuration.
.csproj	Project framework and dependency definitions.

Engineering understanding – How does the application start?

In ASP.NET Core, Program.cs initializes the application, registers required services and configures the HTTP request pipeline.

For example, a typical Razor Pages application uses:

```
None
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddRazorPages();

var app = builder.Build();

app.UseStaticFiles();
app.UseRouting();

app.MapRazorPages();

app.Run();
```

This is an illustrative view of the generated application's startup flow. Do not replace the validated project file with this shortened example.

What happens internally?

1. The application builder initializes configuration and services.
2. Required application services are registered.
3. Middleware is configured to handle incoming HTTP requests.
4. Razor Pages endpoints are mapped.
5. The application starts listening for requests.

Expected outcome: The ASP.NET Core project is created with its application pages, startup configuration and dependencies.

Phase 2 – Build and Run the Application Locally

Concept – Why validate locally?

Before cloud deployment, we should verify that the application compiles, dependencies restore successfully and the application starts correctly.

This separates application-level problems from Azure infrastructure and deployment problems.

Execution

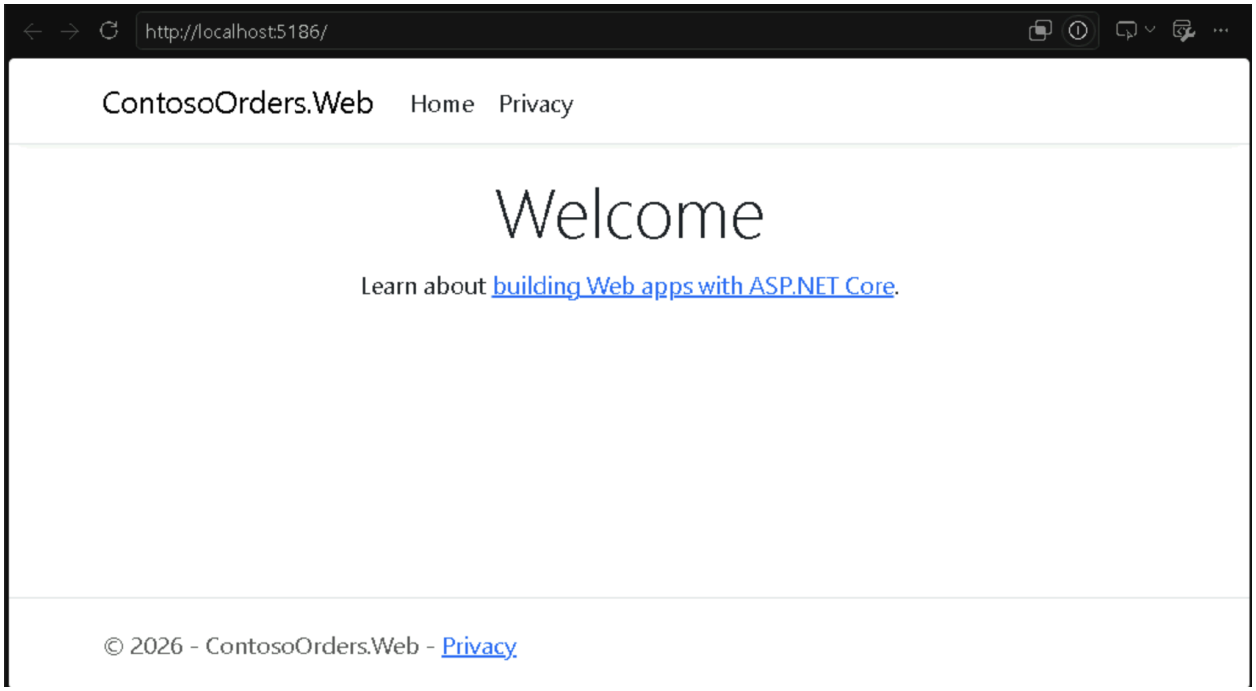
```
None
dotnet restore
dotnet build
dotnet run
```

Open the localhost URL displayed in the terminal, typically:

```
None
https://localhost:7xxx
```

or

```
None
http://localhost:5xxx
```



Validate:

- Homepage loads successfully.
- Privacy page is accessible.
- No startup errors appear in the terminal.
- Build completes without errors.

Engineering understanding

When a user opens the local URL, the browser sends an HTTP request to the ASP.NET Core application. The application processes the request and returns the corresponding web page.

The same basic request-response pattern will continue after deployment to Azure.

Checkpoint 1: The application is running successfully in the local environment.

Phase 3 – Provision the Azure Hosting Environment

Concept – Why do we need cloud hosting?

A locally running application is accessible only through its local development endpoint. To make it available to employees or customers over the internet, we need a cloud-hosting environment.

Azure App Service provides this managed hosting capability.

Execution – Create App Service Plan

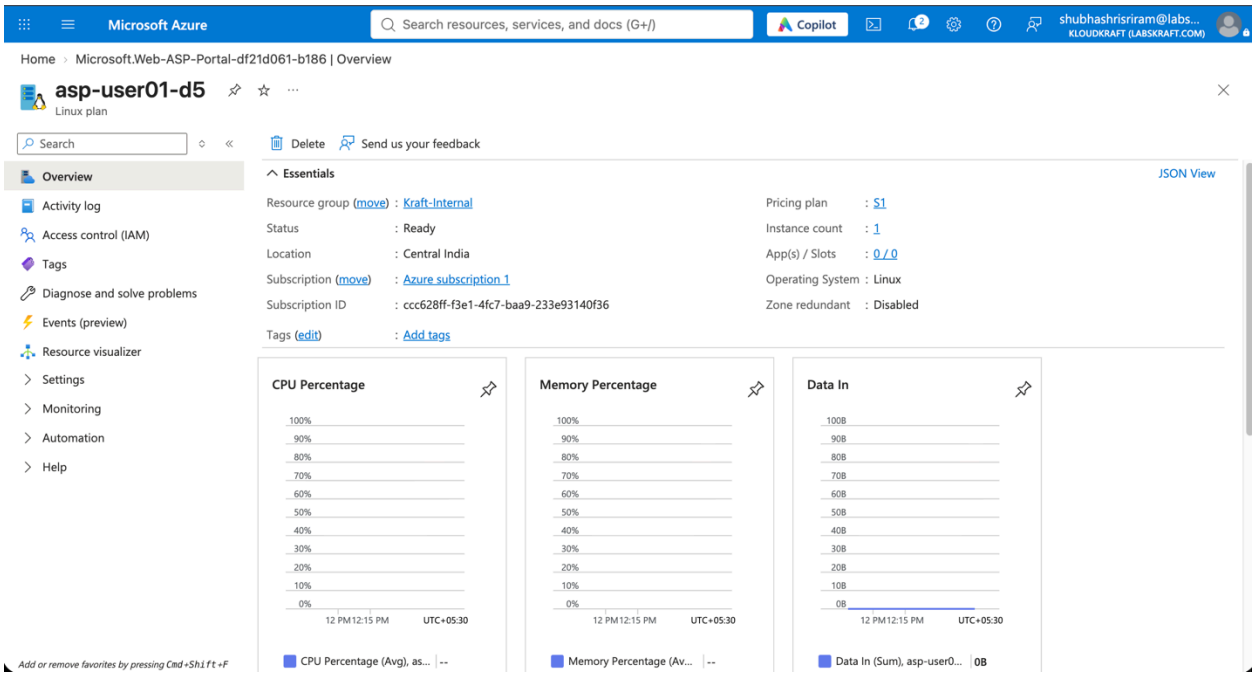
Navigate to:

Azure Portal → App Service Plans → Create.

Configure:

Setting	Value
Subscription	Training subscription
Resource Group	Existing shared RG
Name	asp-user01-d5
Operating System	Linux
Region	Approved training region
Pricing Plan	Standard S1

Select Review + create → Create.



Why are we selecting Standard S1?

The next lab requires deployment slots. We are provisioning the required hosting tier now to avoid recreating the hosting environment later.

Expected outcome: App Service Plan is provisioned with Standard S1 capacity.

Phase 4 – Create the Azure Web App

Concept – What does the Web App provide?

The Web App is the application-hosting resource that provides the public endpoint and executes our deployed ASP.NET Core application.

The App Service Plan supplies its hosting capacity, while the Web App contains our application.

Execution

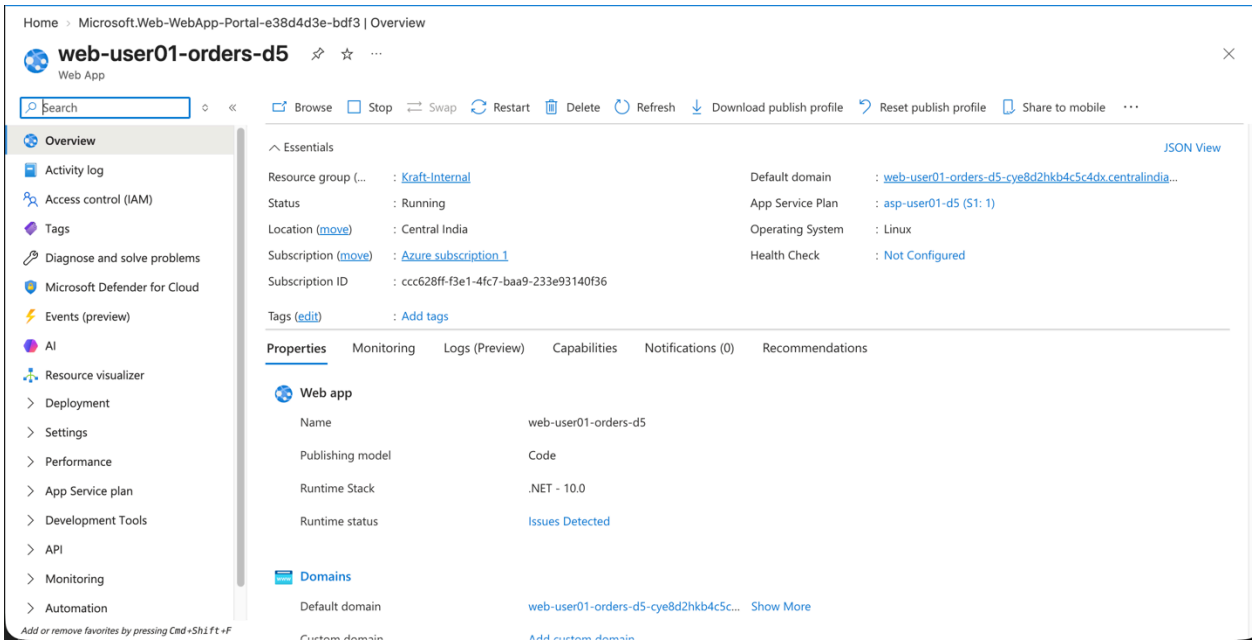
Navigate to:

Azure Portal → App Services → Create → Web App.

Configure:

Setting	Value
Resource Group	Existing shared RG
Name	web-user01-orders-d5
Publish	Code
Runtime Stack	.NET 10 (LTS)
Operating System	Linux
Region	Same as App Service Plan
App Service Plan	asp-user01-d5

Select Review + create → Create.



Engineering understanding

We selected Code publishing because we are deploying an application directly to a managed application-hosting service rather than provisioning and maintaining our own virtual machine.

The selected runtime and operating system establish the execution environment for the application.

Expected outcome: Web App is created and associated with the existing App Service Plan.

Phase 5 – Understand the Web App Endpoint

Navigate to:

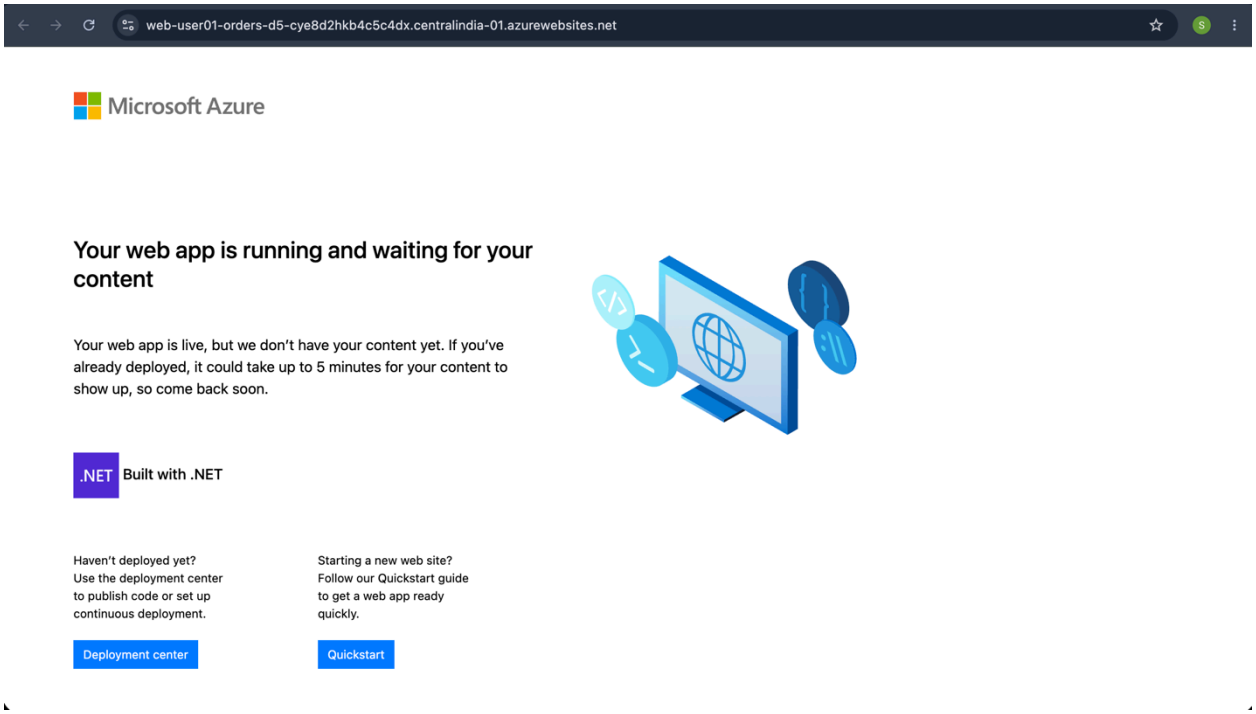
Azure Portal → App Service → Overview.

- Confirm Status is Running.
- Copy the Default domain URL.
- Open the URL in a browser.

Example:

```
None
https://web-user01-orders-d5.azurewebsites.net
```

You may see the default App Service page before deploying your application.



This is expected.

Engineering understanding – What happens when someone opens the URL?

1. The browser sends an HTTPS request to the Web App endpoint.
2. Azure App Service receives the request.
3. The hosted application processes the request.
4. ASP.NET Core generates the appropriate response.
5. The browser displays the returned page.

Expected outcome: The Web App endpoint is available and ready to receive our application deployment.

Phase 6 – Deploy the Application Using VS Code

Concept – Application Deployment

Deployment is the process of publishing application files and required runtime content to the target hosting environment.

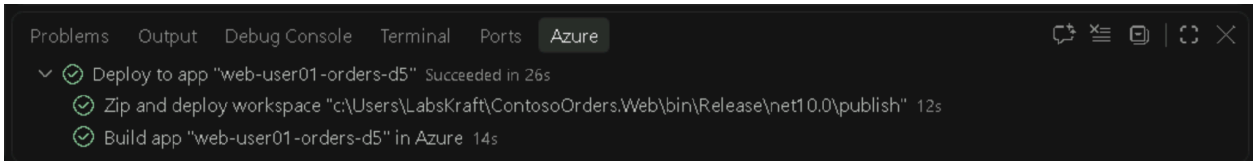
In our implementation, VS Code provides an interface to deploy the local application to Azure App Service.

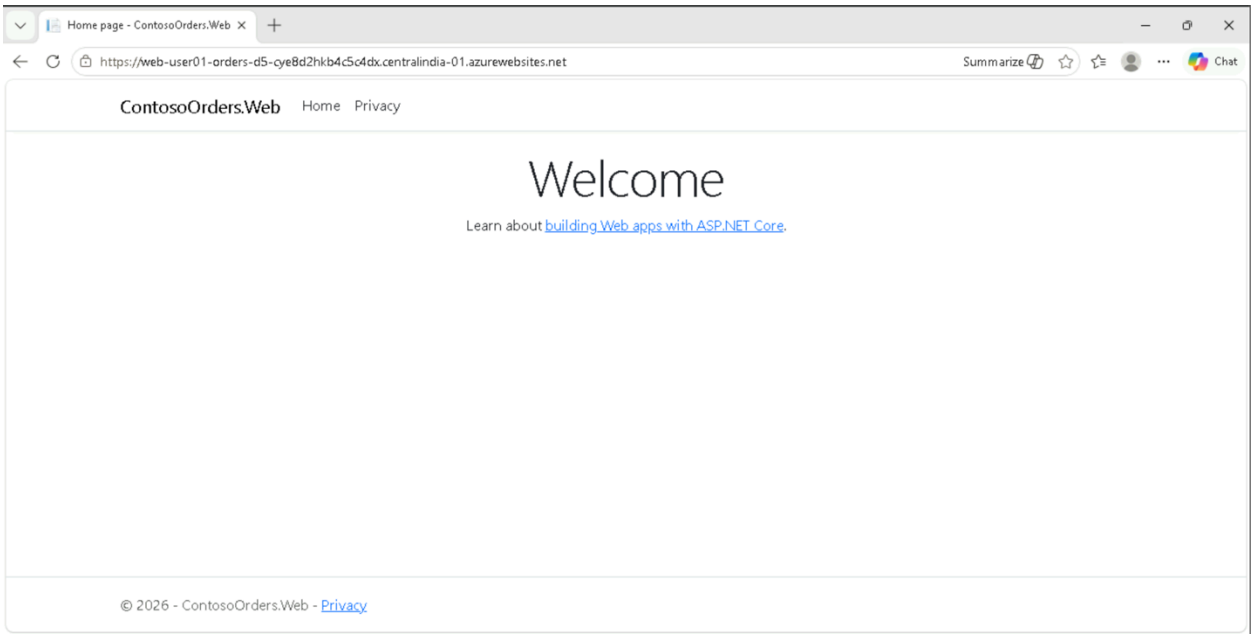
Real-world relevance

Development teams commonly build and test applications locally, then publish them to cloud-hosted environments through approved deployment workflows.

Execution

1. Open VS Code with the ContosoOrders.Web project.
2. Confirm the Azure App Service extension is installed.
3. Open the Azure icon in the VS Code sidebar.
4. Sign in to your Azure subscription.
5. Expand the App Service section.
6. Locate `web-user01-orders-d5`.
7. Right-click the Web App.
8. Select Deploy to Web App.
9. Select the current project folder.
10. Confirm deployment.
11. Wait for the successful deployment notification.
12. Open the Web App URL again.





Engineering understanding

The deployment publishes our application to its Azure hosting destination. App Service then starts the application in its configured environment.

A successful deployment notification confirms the publishing operation, but we must still test the actual application.

Checkpoint 2: The ASP.NET Core application is deployed and accessible through its Azure URL.

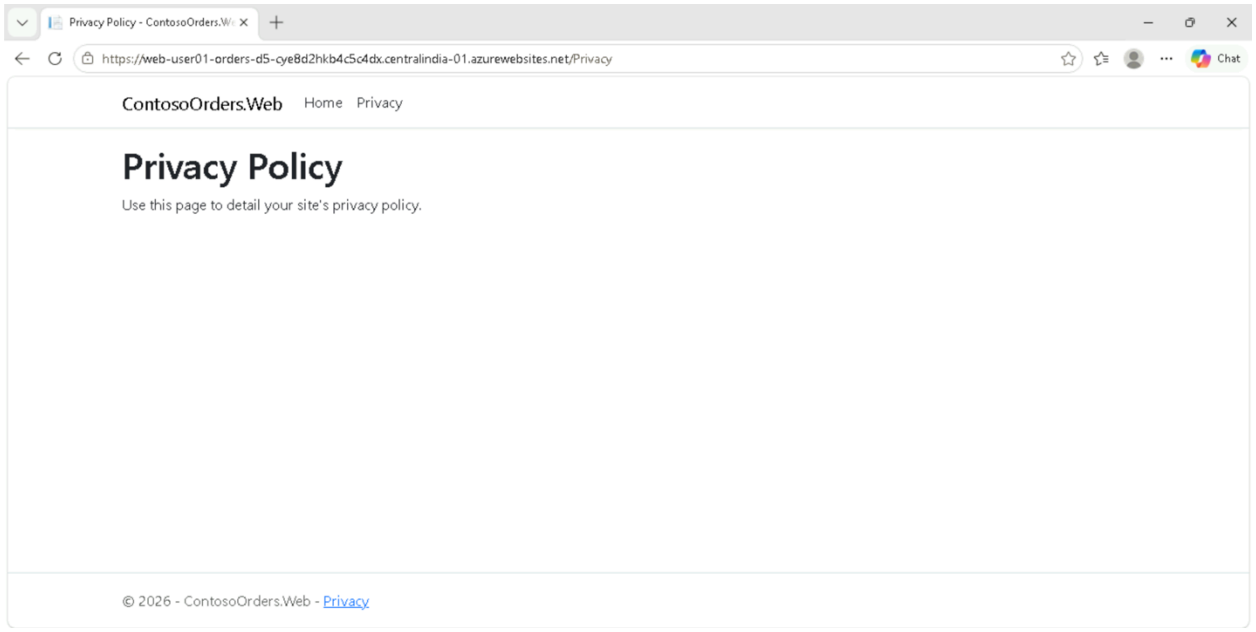
Validate the deployed application

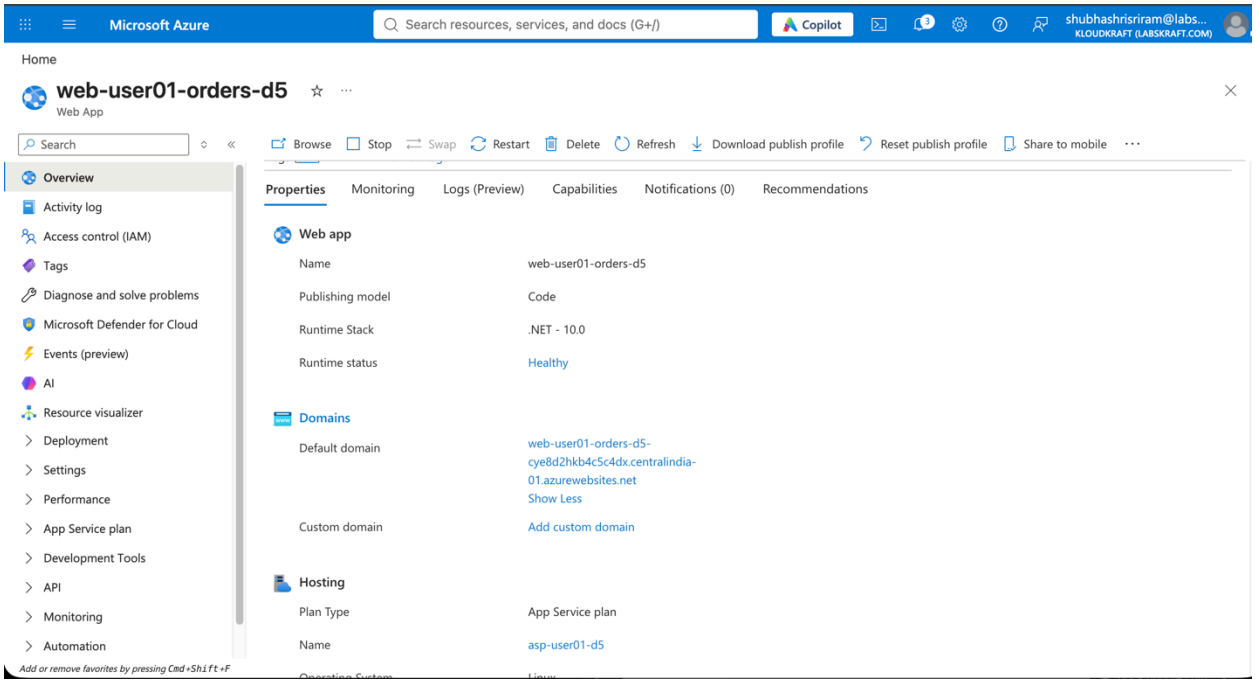
Perform the following checks:

Test	Expected result
Open Web App URL	Website loads

Refresh browser	Website remains accessible
Navigate to Privacy page	Page loads
Application startup	No startup errors
Azure Overview	Status Running

Compare the deployed website with the locally running version.





Phase 7 – Understand Application Configuration

Concept – Why separate configuration from code?

Enterprise applications operate across multiple environments, such as Development, Testing, Staging and Production.

The application code may remain the same, but configuration values can differ between environments.

Azure App Service supports application settings that are exposed to the application as environment variables.

Example

Environment	ENVIRONMENT_NAME
Development	Development
Testing	Testing
Production	Production

This allows teams to change environment-specific configuration without rebuilding the application for every environment.

Execution

Navigate to:

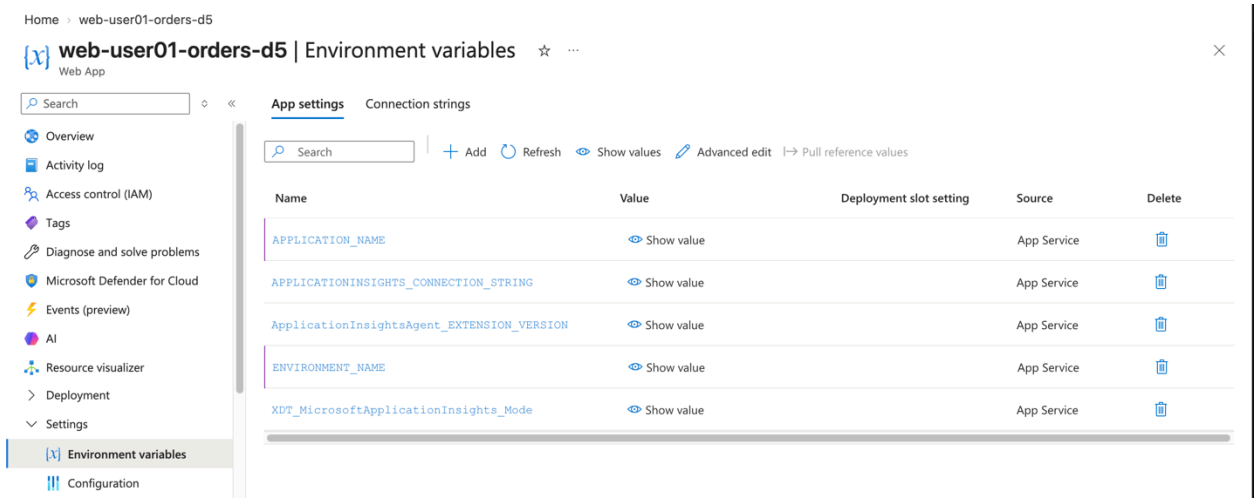
Azure Portal → App Service → Settings → Environment variables.

Under App settings, select Add.

Create:

Name	Value
APPLICATION_NAME	Contoso Orders
ENVIRONMENT_NAME	Day5-Training

Select Apply and save.



App Service may restart after configuration changes.

Coding understanding – How does ASP.NET Core read a setting?

ASP.NET Core provides the IConfiguration interface for accessing application configuration.

For example:

```
None
var appName =
    builder.Configuration["APPLICATION_NAME"];

var environmentName =
    builder.Configuration["ENVIRONMENT_NAME"];
```

These values can be accessed by application components through configuration injection.

Important: The current lab prepares these settings for subsequent labs. We are not modifying the validated application to display or consume them at this stage.

Expected outcome: Both application settings are saved in the Web App configuration.

Phase 8 – Application Validation and Troubleshooting

Concept – Deployment versus Application Health

A deployment can complete successfully while the application itself fails to start or process requests.

In production, operations engineers must distinguish deployment failures, startup failures, configuration problems and runtime application errors.

Execution – Validate the deployed application

Test	Expected Result
Open Web App URL	Website loads
Refresh browser	Website remains accessible
Navigate to Privacy page	Page loads
Application startup	No startup errors
Azure Overview	Status Running

Debugging and troubleshooting

Symptom	What to investigate

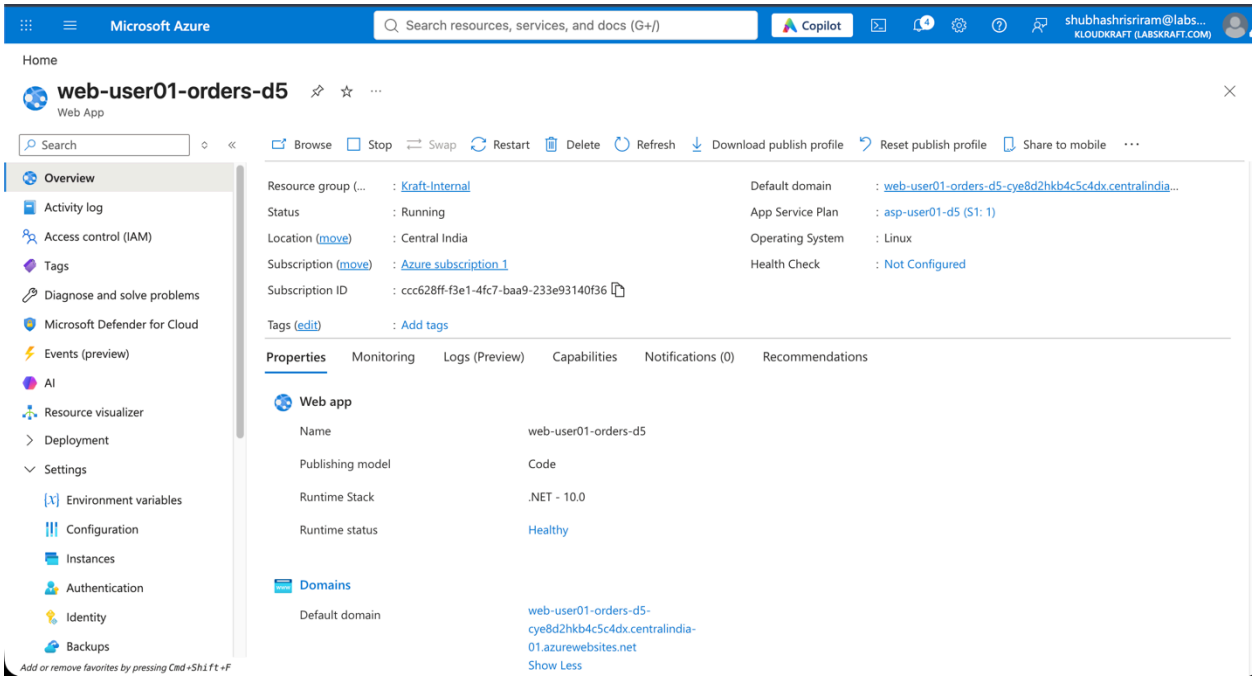
Website unavailable	Web App status, URL and hosting configuration.
HTTP 404	Requested route, page mapping and URL.
HTTP 500	Application exceptions and startup/runtime logs.
Application fails to start	Runtime configuration and application startup errors.
Old application version appears	Deployment status and browser caching.
Configuration changes not reflected	Setting name, saved configuration and application restart.

Where do we investigate?

Azure Portal → App Service → Monitoring.

Overview

- Check application status.
- Verify the default domain.



Log Stream

- Review available application output.
- Inspect startup and runtime messages.
- Investigate application errors.

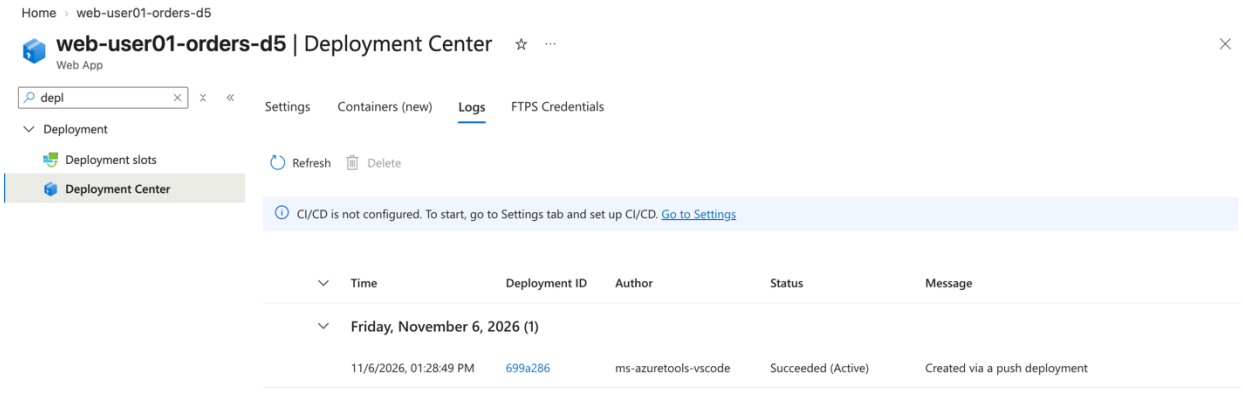


TIMESTAMP (UTC)	TYPE	LEVEL	CONTAINER	MESSAGE
08:09:52.959	Platform	INFO	web-user01-orders-d5	State: Starting, Action: WarmUpProbeSucceeded, LastError: , Las…
08:09:53.966	Platform	INFO	web-user01-orders-d5	Site is running with deployment version: 699a2869-c3fe-45bf-a1d…
08:09:53.974	Platform	INFO	web-user01-orders-d5	Site started.
08:09:53.976	Platform	INFO	web-user01-orders-d5	State: Started, Action: None, LastError: , LastErrorTimestamp: …
08:09:53.984	Platform	INFO	web-user01-orders-d5	Site is running with patch version DOTNETCORE-10.0.14
08:10:14.191	Platform	INFO	web-user01-orders-d5	State: Stopping, Action: StoppingSite, LastError: , LastErrorTi…
08:10:14.199	Platform	INFO	web-user01-orders-d5	State: Stopping, Action: StoppingSiteContainers, LastError: , L…
08:10:14.200	Platform	INFO	web-user01-orders-d5	Container is terminating. Grace period: 5 seconds.
08:10:14.221	Platform	INFO	web-user01-orders-d5	Stop and delete container. Retry count = 0
08:10:14.221	Platform	INFO	web-user01-orders-d5	Stopping container: 8173184d076a_web-user01-orders-d5.
08:10:19.384	Platform	INFO	web-user01-orders-d5	Deleting container: 8173184d076a_web-user01-orders-d5. Retry co…
08:10:20.252	Platform	INFO	web-user01-orders-d5	Container spec TerminationMessagePolicy path
08:10:20.259	Platform	INFO	web-user01-orders-d5	Container is terminated. Total time elapsed: 6052 ms.
08:10:20.259	Platform	INFO	web-user01-orders-d5	Site: web-user01-orders-d5 stopped.

101 of 144 entries

Deployment Center

- Review deployment history if available.
- Check the latest deployment result.



Add or remove favorites by pressing Cmd+Shift+F

Local debugging

When an application fails locally, inspect:

- Terminal startup output.
- Build errors.
- Dependency restoration errors.
- Application exceptions.

Azure debugging

When an application works locally but fails after deployment, investigate:

- Runtime stack and operating system.
- Application startup logs.
- Deployment status.
- Environment-specific settings.
- Hosting configuration.

Expected outcome: Application accessibility is confirmed and basic monitoring and troubleshooting locations are understood.

5. Evidence Collection

Capture:

- Local project structure in VS Code.
- Successful dotnet build.
- Application running locally.
- App Service Plan showing Standard S1.
- Web App configuration showing .NET 10.
- Successful VS Code deployment.
- Deployed website in browser.
- App Service Overview showing Running.
- Application settings configuration.

6. Completion Criteria

Validation	Expected Result
Project creation	Successful
.NET 10 restore	Successful
Build	Zero errors
Local execution	Website loads
App Service Plan	Standard S1
Web App	Running
Deployment	Successful
Public URL	Accessible
Application settings	Saved
Monitoring	Basic checks completed

7. Production Engineering Takeaways

At the end of this lab, participants should understand:

- Why organizations use Azure App Service instead of managing their own web-hosting virtual machines.
- The difference between an App Service Plan and a Web App.
- How an ASP.NET Core application processes browser requests.
- How applications are deployed from development environments to Azure.
- Why application configuration is separated from source code.
- How to distinguish deployment issues from application runtime issues.
- Where to investigate application health and startup errors.

Real-world scenario

Imagine your organization has developed an internal employee portal.

The development team builds the application using ASP.NET Core. The cloud operations team provisions an App Service Plan and Web App, publishes the application, configures the environment and monitors its availability.

If the portal becomes unavailable, the support engineer investigates the Web App status, application logs, deployment history and runtime configuration before escalating to the development team.

This is the operational responsibility demonstrated in this lab.

8. Connection to Lab 5.2

Our application currently hosts the web interface, but the next requirement is to process customer orders without making users wait for the complete processing operation.

In Lab 5.2, we will introduce:

- Azure Storage Account and Queue Storage.
- Asynchronous message processing concepts.
- Azure Managed Identity for passwordless authentication.
- Queue message publishing from our ASP.NET Core application.
- Deployment slots for controlled application releases.

The existing Web App and App Service Plan will be reused.

9. Cleanup

Do not delete the following resources:

- App Service Plan – asp-user01-d5
- Web App – web-user01-orders-d5

Both are required for Lab 5.2.